BÀI TẬP TÌM KIẾM TRONG AI

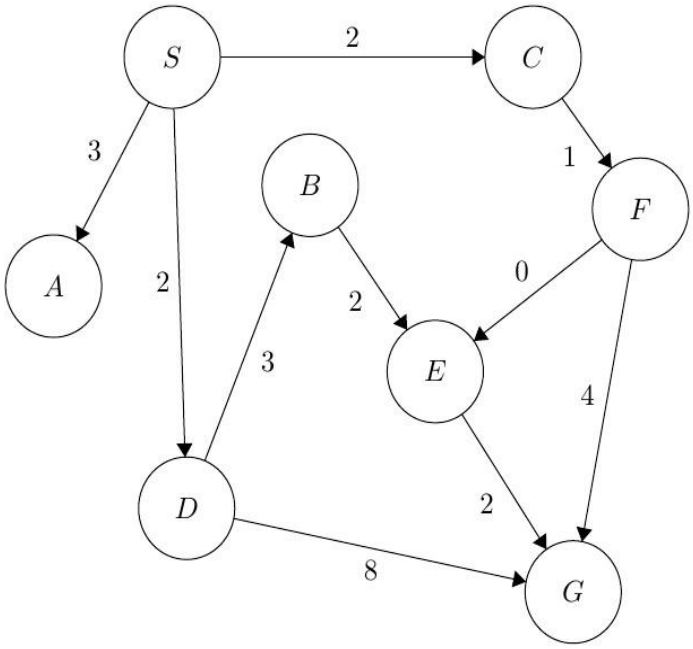
Họ và tên	Trương Văn Thiện
MSSV / Lớp	3123410356/DCT1232

Mục tiêu học tập

- Mô phỏng các thuật toán BFS, DFS và Iterative Deepening.
- Biểu diễn bài toán Tháp Hà Nội dưới dạng bài toán tìm kiếm.
- Thiết kế và phân tích heuristic cho A*.
- So sánh DFS, BFS, UCS, Greedy Search và A* trên cùng một bài toán.
- Phân tích các phát biểu lý thuyết về heuristic, A*, Minimax và Expectimax.

Bài 1. Các thuật toán tìm kiếm

Cho đồ thị có trạng thái bắt đầu S và trạng thái đích G như hình dưới đây.



Hình 1. Đồ thị dùng cho các thuật toán tìm kiếm

Yêu cầu: Với mỗi thuật toán, hãy ghi thứ tự các nút được đưa vào explored set, đường đi được trả về và tổng chi phí của đường đi. Nếu có hòa, phá hòa theo thứ tự alphabet của trạng thái cuối cùng trong đường đi.

1.1. Breadth-First Search (BFS)

Nội dung	Kết quả
Explored Set	S, A, C, D, F, B, G

Path	S → D → G
Total Cost	2 + 8 = 10

1.2. Depth-First Search (DFS)

Nội dung	Kết quả
Explored Set	S, A, D, G
Path	S → A → D → G
Total Cost	3 + 3 + 8 = 14

1.3. Iterative Deepening Search (IDS)

Giới hạn độ sâu	Các nút được duyệt
0	Duyệt S. Không tìm thấy.
1	Duyệt S, A, C, D. Không tìm thấy.
2	Duyệt S, A, D, B, G. Tìm thấy G.
...	

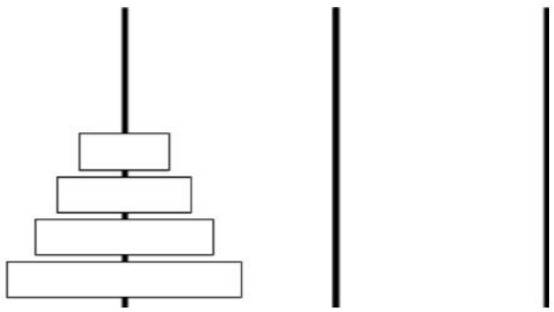
Path cuối cùng: S→ D→G

Total Cost: 10

Câu hỏi thảo luận: BFS duyệt theo chiều rộng, đảm bảo tìm thấy đường đi ngắn nhất (số cạnh) nhưng tốn bộ nhớ. DFS duyệt theo chiều sâu, tiết kiệm bộ nhớ nhưng không đảm bảo tối ưu và có thể bị kẹt trong nhánh sâu vô hạn.

Bài 2. Tháp Hà Nội - Tower of Hanoi

Bài toán gồm n đĩa có kích thước khác nhau và ba cọc. Ban đầu các đĩa được xếp trên một cọc theo thứ tự từ lớn đến nhỏ. Mỗi lần chỉ được di chuyển một đĩa và không được đặt đĩa lớn lên trên đĩa nhỏ. Mục tiêu là chuyển toàn bộ đĩa sang cọc thứ ba.



Hình 2. Trạng thái ban đầu của bài toán Tháp Hà Nội

2.1. Biểu diễn trạng thái

Một trạng thái có thể được biểu diễn bằng một bộ ba (A, B, C), trong đó A, B, C là danh sách các đĩa trên mỗi cọc, được sắp xếp từ trên xuống dưới (hoặc từ nhỏ đến lớn). Ví dụ: ((n, n-1, ..., 1), (), ()) là trạng thái ban đầu với tất cả đĩa trên cọc A.

2.2. Kích thước không gian trạng thái

Với n đĩa, mỗi đĩa có 3 lựa chọn cọc để nằm. Do đó, tổng số trạng thái có thể có là $\mathbf{3^n}$.

2.3. Trạng thái bắt đầu

S0 = ((n, n-1, ..., 1), (), ()) (Tất cả đĩa nằm trên cọc A, xếp từ lớn đến nhỏ).

2.4. Các hành động hợp lệ

Từ một trạng thái bất kỳ, một hành động hợp lệ là di chuyển đĩa trên cùng từ cọc nguồn sang cọc đích nếu:

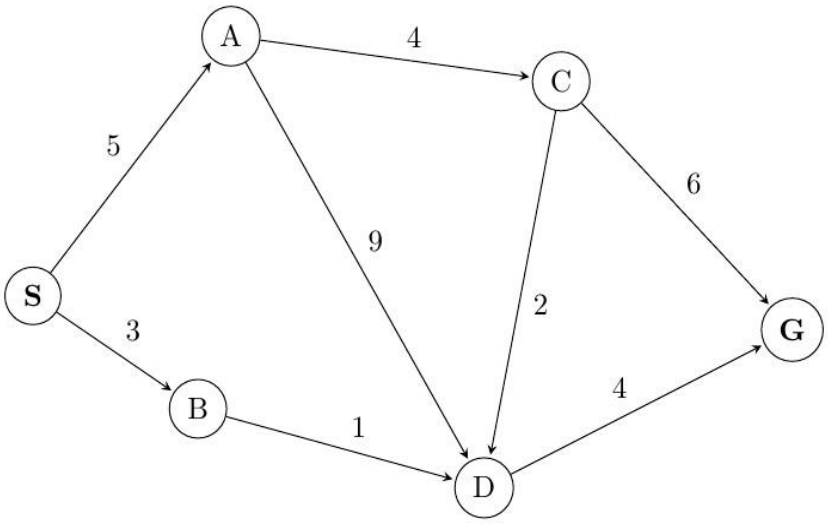
- Cọc nguồn không rỗng.
- Cọc đích rỗng hoặc đĩa trên cùng của cọc đích có kích thước lớn hơn đĩa đang được di chuyển

2.5. Goal Test

Hàm *goalTest(state)* trả về True khi tất cả
các đĩa nằm ở cọc thứ 3 (cọc C).

Bài 3. Thiết kế và phân tích Heuristic

Xét đồ thị dưới đây. Hãy thiết kế heuristic và phân tích ảnh hưởng của heuristic đến thuật toán A*.



Hình 3. Đồ thị dùng để thiết kế heuristic

Công thức A*: f(n) = g(n) + h(n), trong đó g(n) là chi phí thực tế từ S đến n và h(n) là chi phí ước lượng từ n đến đích.

3.1. Thiết kế heuristic admissible và consistent

Điền giá trị h(n) cho từng nút sao cho heuristic phù hợp với yêu cầu của nhóm.

Node	h(n)
S	8 (S→B→D→G)
A	10 (A→C→G hoặc A→C→D→G)
B	5 (B→D→G)
C	6 (C→G hoặc C→D→G)
D	4 (D→G)
G	0

3.2. Chạy A* với heuristic đã thiết kế

Explored Set: S, B, D, G

Path: S → B → D → G

Total Cost: 3 + 1 + 4 = 8

3.3. Heuristic admissible nhưng không consistent

Đề xuất bộ giá trị: h(S)=8, h(B)=4, h(C)=5, h(D)=4, h(A)=9, h(G)=0.

- **Kiểm tra admissible:** Tất cả đều <= chi phí thực tế (Ví dụ: h(B)=4 <= 5, h(S)=8 <= 8).
- **Cạnh vi phạm consistency:** Xét cạnh S -> B:
  - h(S) = 8
  - c(S, B) + h(B) = 3 + 4 = 7
  - Vì 8 > 7, nên h(S) <= c(S,B) + h(B) bị vi phạm. Vậy heuristic này admissible nhưng không consistent.

3.4. Câu hỏi mở rộng

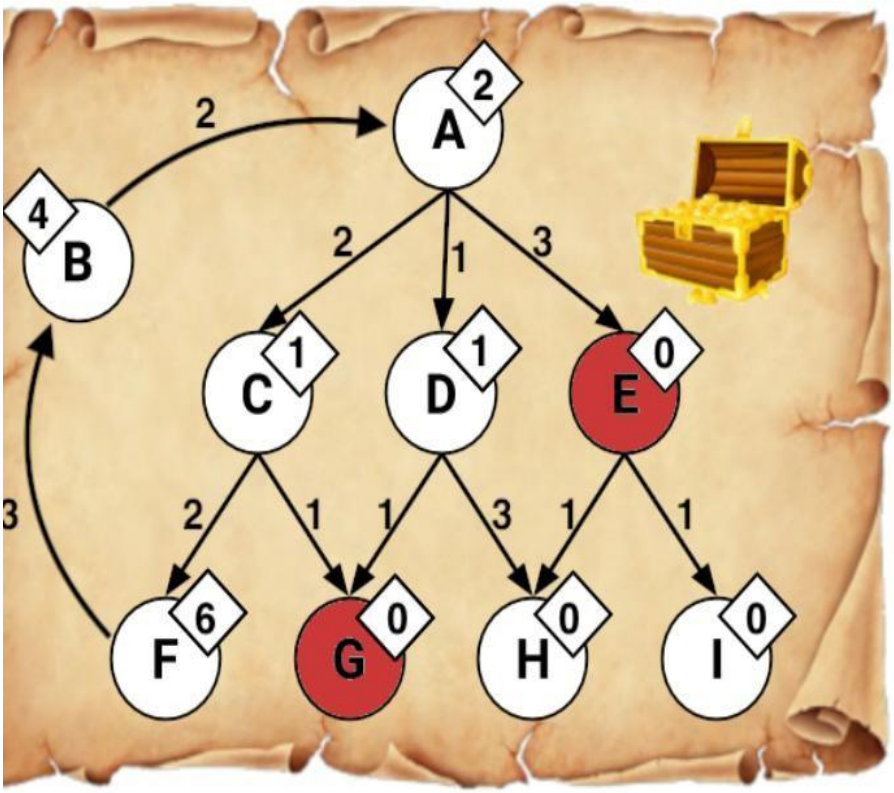
Giải thích tại sao consistent => admissible:
Giả sử h là consistent, tức là h(n) <= c(n, n') + h(n'). Bằng quy nạp theo số bước từ n đến đích G:

- Nếu n = G, h(G) = 0 <= 0 (đúng).
- Giả sử đúng với n', ta có: h(n) <= c(n, n') + h(n') <= c(n, n') + chi phí thực tế từ n' đến G = chi phí thực tế từ n đến G.

- Vậy h(n) <= h*(n), do đó consistent => admissible.

Bài 4. Treasure Hunting (Tìm kho báu)

A là đảo xuất phát. Các nút màu đỏ là các đảo có kho báu; chỉ cần tới một trong các đảo đó. Mũi tên biểu diễn hành động có hướng; số trên cạnh là action cost. Giá trị trong hình thoi là heuristic ước lượng chi phí tối thiểu từ nút hiện tại tới một kho báu.



Hình 4. Bản đồ tìm kho báu (giữ nguyên hình gốc)

Quy tắc phá hòa: khi lấy phần tử khỏi frontier, nếu hòa thì chọn theo thứ tự alphabet.

4.1. Depth-First Search

Explored Set: A, B, F, C, G

Path returned: A → C →G (Hoặc A → B → F -> C→ G, nhưng DFS ưu tiên alphabet nên lấy B trước, rồi F, rồi C, rồi G).

4.2. Breadth-First Search

Explored Set: A, B, C, D, E, F,
G

Path returned: A → C →G

4.3. Uniform-Cost Search

Explored Set: A, D, B, C, G

Path returned: A → D →G

Cost: 2

4.4. Greedy Search

Explored Set: A, E

Path returned: A →E

Cost: 3

4.5. A* Search

Explored Set: A, D,
G

Path returned: A→D→G

Cost: 2

Bảng tổng hợp so sánh

Thuật toán	Tiêu chí chọn node	Path	Cost
DFS	Độ sâu	A → C →G	3
BFS	Độ sâu nhỏ nhất	A → C →G	3
UCS	g(n)	A → D →G	2
Greedy	h(n)	A →E	3
A*	g(n)+h(n)	A→D→G	2

Bài 5. Câu hỏi Đúng/Sai

Với mỗi phát biểu, hãy ghi Đúng hoặc Sai và giải thích ngắn gọn. Nếu sai, nên đưa ra phản ví dụ.

Câu 1. Depth-first search luôn mở rộng ít nhất nhiều node bằng A* sử dụng một admissible heuristic. Đúng / Sai: **Sai**

Giải thích: DFS có thể mở rộng nhiều node hơn A* nếu nó đi vào một nhánh sâu không liên quan đến đích, trong khi A* sử dụng heuristic để hướng đến đích. A* thường mở rộng ít node hơn nhờ tính định hướng.

Câu 2. Giả sử một quân xe trong cờ vua có thể đi bất kỳ số ô theo hàng hoặc cột trong một nước đi và không thể nhảy qua quân khác. Manhattan distance là một admissible heuristic cho số nước đi nhỏ nhất từ ô A đến ô B. Đúng / Sai: **Đúng**

Giải thích: Manhattan distance là khoảng cách ngắn nhất về mặt hình học (số hàng + số cột). Vì quân xe không thể đi xuyên qua vật cản, số nước đi thực tế luôn lớn hơn hoặc bằng khoảng cách này. Do đó, nó không bao giờ đánh giá quá cao chi phí thực tế (admissible).

Câu 3. Euclidean distance là một admissible heuristic cho bài toán tìm đường của

Pacman. Đúng / Sai: **Đúng**

Giải thích: Euclidean distance là khoảng cách đường thẳng giữa hai điểm, luôn nhỏ hơn hoặc bằng khoảng cách thực tế mà Pacman phải đi (do có tường chắn). Do đó, nó không bao giờ đánh giá quá cao chi phí thực tế.

Câu 4. Tổng của nhiều admissible heuristics vẫn luôn là một admissible heuristic.

Đúng / Sai: **Sai**

Giải thích: Tổng của các admissible heuristics có thể lớn hơn chi phí thực tế. Ví dụ: $h1(n) = 2$, $h2(n) = 3$, nhưng chi phí thực tế từ n đến đích là 4. Tổng $h1+h2 = 5 > 4$, vi phạm tính admissible. (Trừ khi ta dùng max, nhưng tổng thì không).

Câu 5. Admissibility của một heuristic cho A* kéo theo consistency.

Đúng / Sai: **Sai**

Giải thích: Admissibility không kéo theo consistency. Một heuristic có thể admissible nhưng không consistent (như ví dụ ở Bài 3.3). Tính consistent mạnh hơn tính admissible.

Câu 6. A* với graph search luôn tối ưu nếu heuristic admissible.

Đúng / Sai: **Sai**

Giải thích: A* với graph search chỉ tối ưu nếu heuristic là **consistent**. Nếu heuristic chỉ admissible mà không consistent, A* với graph search có thể bỏ qua đường đi tối ưu vì nó đánh dấu nút đã duyệt (explored) và không cập nhật lại khi tìm thấy đường đi tốt hơn.

Tổng kết nhanh

Nhóm	Thuật toán	Hàm/tiêu chí chính
Uninformed Search	BFS / DFS / IDS	Không sử dụng heuristic
Cost-based Search	UCS	g(n)
Informed Search	Greedy	h(n)
Informed Search	A*	g(n)+h(n)

Ghi nhớ: A* dùng f(n) = g(n) + h(n).

---o0o---

(Hết)